

STAT 432: Homework 05

Jacob Jacobson
September 26, 2026

Question 1: Estimate the bias and variance of KNN

(a) **One training dataset.** At the requested target,

$$f(\mathbf{x}_0) = 0.5(0.5) + \sin(0.7) - 0.3(1)^2 = 0.594218.$$

I used seed 43251. The predictions from the first generated training set were:

k	1	3	5	7	9	11	13
Prediction	.7888	.6622	.6643	.6100	.6319	.4925	.5223
k	15	17	19	21	23	25	27
Prediction	.5810	.5738	.5751	.5801	.5519	.5215	.4953
k	29						
Prediction	.4973						

(b) **Repeated simulation.** I independently regenerated both the training covariates and responses 200 times, using the same realized dataset for all k within a repetition. For column k of the resulting 200×15 prediction matrix, I calculated

$$\widehat{\text{Bias}}_k = \widehat{f}_k - f(\mathbf{x}_0), \quad \widehat{\text{Var}}_k = \frac{1}{200} \sum_{s=1}^{200} (\widehat{f}_{sk} - \widehat{f}_k)^2.$$

k	Bias	Squared bias	Variance	MSE
1	-.00125	.000002	.037181	.037183
3	-.02936	.000862	.019832	.020694
5	-.04082	.001666	.014731	.016397
7	-.04559	.002078	.010866	.012944
9	-.05187	.002690	.009467	.012158
11	-.06236	.003889	.009506	.013395
13	-.06780	.004597	.009398	.013994
15	-.07066	.004993	.008542	.013536
17	-.08150	.006642	.008214	.014856
19	-.08939	.007990	.008430	.016420
21	-.09608	.009232	.008413	.017645
23	-.10280	.010569	.008103	.018672
25	-.10785	.011631	.008091	.019722
27	-.11443	.013094	.007778	.020872
29	-.12169	.014808	.007475	.022283

The largest absolute discrepancy in $\widehat{\text{MSE}} - \widehat{\text{Bias}}^2 - \widehat{\text{Var}}$ was 6.77×10^{-17} , so the empirical decomposition holds to numerical precision.

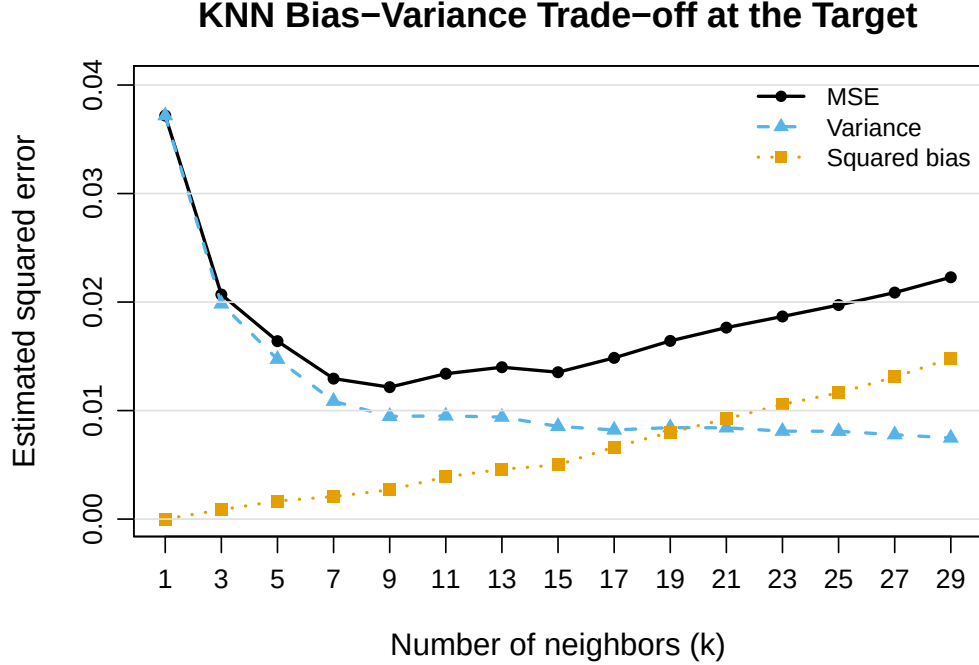


Figure 1: Bias–variance decomposition for the estimated mean response.

Increasing k averages more responses and generally reduces variance, but it also admits points farther from $\mathbf{x}_0$, increasing smoothing bias. Here MSE is minimized at $k = 9$ among the tested values. The small nonmonotonic movements in estimated variance are reasonable with only 200 repetitions. Increasing k resembles increasing the penalty in ridge or lasso because both reduce flexibility and usually exchange variance for bias. The mechanisms differ: KNN enlarges a local neighborhood, ridge shrinks all slopes continuously, and lasso can set slopes exactly to zero. Because the covariates are regenerated too, KNN’s variance includes changes in neighbor locations as well as response noise and is not simply $0.1^2/k$.

Question 2: Compare KNN with lasso

I used seed 43252. Within each setting, 5NN and lasso used the same 400 training and 1,000 test observations. For each cross-validation fold I estimated predictor means and population-form scales from that fold’s training portion only. I centered its training response, fit the full descending grid of 41 penalties by coordinate descent for

$$\frac{1}{2n} \|\tilde{\mathbf{y}} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \lambda \|\boldsymbol{\beta}\|_1,$$

and added the fold-specific training response mean back to validation predictions. After selecting λ , I repeated these transformations using all 400 training cases and applied them unchanged to the test set.

Setting	5NN test MSE	Lasso test MSE	Selected λ	Nonzero slopes
Independent	.5565	.2602	.02512	11
Correlated	.1609	.2260	.01000	16

(b) Effect of correlation. With independent predictors, the 27 response-irrelevant coordinates contribute noise to Euclidean distance. Thus a point that is close in all 30 coordinates need not be especially

close in the three coordinates governing the response, and 5NN performs poorly. Under strong positive correlation, much of the 30-dimensional variation is shared; proximity across all coordinates is more informative about the relevant coordinates in this realization, and 5NN improves substantially.

Lasso does not use all predictors symmetrically as KNN does. It estimates a separate slope for each standardized column and sets some slopes to zero. In correlated data, even an irrelevant variable can proxy for a relevant variable, and correlated variables can compete for inclusion. That helps explain why 16 slopes remain nonzero here even though only the first three variables directly enter the data-generating response.

(c) Functional form and scope of the comparison. The lasso model includes only linear terms. It can approximate $\sin(X_2)$ over the range where sine is roughly linear, and its intercept absorbs the mean of $-0.3X_3^2$, but it cannot reproduce the changing quadratic effect. KNN does not impose this linear restriction, although it pays for that flexibility when distance contains many irrelevant directions. Therefore the table mixes two issues: neighborhood quality and lasso's functional-form bias. Two particular simulated datasets do not justify claiming that correlation always helps KNN or that either procedure is always superior; the outcome depends on covariance structure, sample size, signal, noise, k , penalty selection, and how well each model class represents the regression function.

Question 3: Observed dimension and latent dimension

I used seed 43254. For each dataset I generated one matrix $\mathbf{A}$, then generated its training and test observations with that same matrix. The fitted model received only the 100 observed coordinates.

(a) One dataset per latent dimension.

k	MSE, $m = 3$	MSE, $m = 30$	k	MSE, $m = 3$	MSE, $m = 30$
2	1.449	3.457	42	1.506	2.561
10	1.253	2.583	50	1.590	2.563
18	1.293	2.533	58	1.660	2.594
26	1.380	2.524	66	1.740	2.607
34	1.448	2.512	74	1.817	2.651
			82	1.872	2.673

(b) Average of 50 complete experiments.

k	MSE, $m = 3$	MSE, $m = 30$	k	MSE, $m = 3$	MSE, $m = 30$
2	1.635	3.063	42	1.496	2.456
10	1.288	2.350	50	1.555	2.492
18	1.321	2.346	58	1.615	2.527
26	1.373	2.378	66	1.674	2.556
34	1.438	2.413	74	1.737	2.589
			82	1.800	2.619

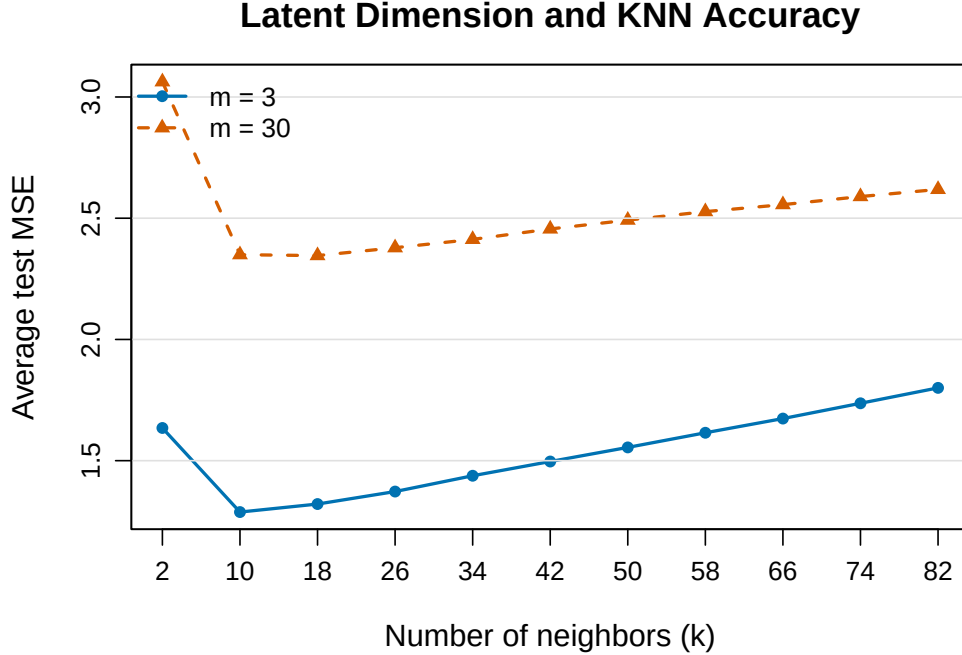


Figure 2: Average test MSE across 50 independently generated datasets.

The $m = 3$ setting is consistently easier. Its average curve is minimized at $k = 10$ (MSE 1.288), whereas the $m = 30$ curve reaches its lowest displayed value near $k = 18$ (MSE 2.346). These are controlled simulation comparisons, not cross-validation estimates used to select a final fitted model.

(c) Why latent dimension matters. Although both settings expose 100 coordinates and Y depends only on Z_1 and Z_2 , the distance has different information content. When $m = 3$, most structured variation in $\mathbf{X}$ is generated by three latent directions. When $m = 30$, Euclidean distance also responds to 28 latent variables absent from the response. Observations close in (Z_1, Z_2) can then be far apart along irrelevant latent directions, so the selected neighbors are less useful for predicting Y .

Measurement noise $\boldsymbol{\eta}_i$ independently perturbs all 100 coordinates and further disrupts neighbor rankings. Without this term, $\mathbf{A}\mathbf{Z}_i$ lies in the column space of $\mathbf{A}$, with dimension at most m . Adding full-dimensional Gaussian noise means $\mathbf{X}_i$ is not confined exactly to that subspace; the model has a lower-dimensional signal embedded in noisy 100-dimensional observations.

Question 4: Classify handwritten digits

(a) Direct 5NN implementation. I used the first 1,000 training rows and first 500 test rows, left the pixel values unstandardized, and excluded the label column. The following core function computes every distance directly. R's `order` resolves equal distances by the supplied training-row index, and `which.max` returns the smallest digit in a voting tie.

```
knn_classify <- function(X, y, X_test, metric) {
  prediction <- integer(nrow(X_test))
  for (i in seq_len(nrow(X_test))) {
    delta <- sweep(X, 2, X_test[i, ], "-")
    distance <- if (metric == "euclidean") {
      sqrt(rowSums(delta^2))
    } else {

```

```

    rowSums(abs(delta))
  }
  neighbors <- order(distance, seq_len(nrow(X)))[1:5]
  prediction[i] <-
    which.max(tabulate(y[neighbors] + 1, nbins = 10)) - 1
}
prediction
}

```

(b) **Test errors and confusion matrices.** Euclidean distance made 61 errors, for a test error of 0.122; Manhattan distance made 69 errors, for a test error of 0.138. Rows below are predicted digits and columns are true digits.

<i>Euclidean distance</i>										
Pred.\True	0	1	2	3	4	5	6	7	8	9
0	118	0	7	1	0	4	2	0	2	0
1	0	66	0	0	2	0	0	0	0	1
2	0	0	39	1	0	0	2	0	0	1
3	0	0	0	23	0	4	0	0	2	0
4	0	0	1	0	27	1	1	1	0	1
5	0	0	0	1	0	13	0	0	0	0
6	0	2	0	0	0	1	41	0	2	0
7	0	0	0	0	1	1	0	34	0	4
8	0	0	3	2	0	2	0	0	37	0
9	1	0	0	1	4	0	0	1	1	41

<i>Manhattan distance</i>										
Pred.\True	0	1	2	3	4	5	6	7	8	9
0	118	0	6	1	0	5	3	0	4	0
1	0	67	2	0	2	0	1	0	0	1
2	0	0	35	0	0	0	2	0	0	0
3	0	0	1	24	0	4	0	0	2	0
4	0	0	1	1	27	1	0	1	0	1
5	0	0	0	1	0	11	0	0	0	0
6	0	1	0	0	0	2	40	0	2	0
7	0	0	2	1	1	0	0	34	0	5
8	0	0	2	0	0	3	0	0	34	0
9	1	0	1	1	4	0	0	1	2	41

For both metrics, the most common single confusion is a true 2 predicted as 0 (seven Euclidean cases and six Manhattan cases). Other recurring errors include true 5s predicted as 0 or 3, true 9s predicted as 7, and true 4s predicted as 9.

(c) **Inspecting Euclidean errors and comparing metrics.**

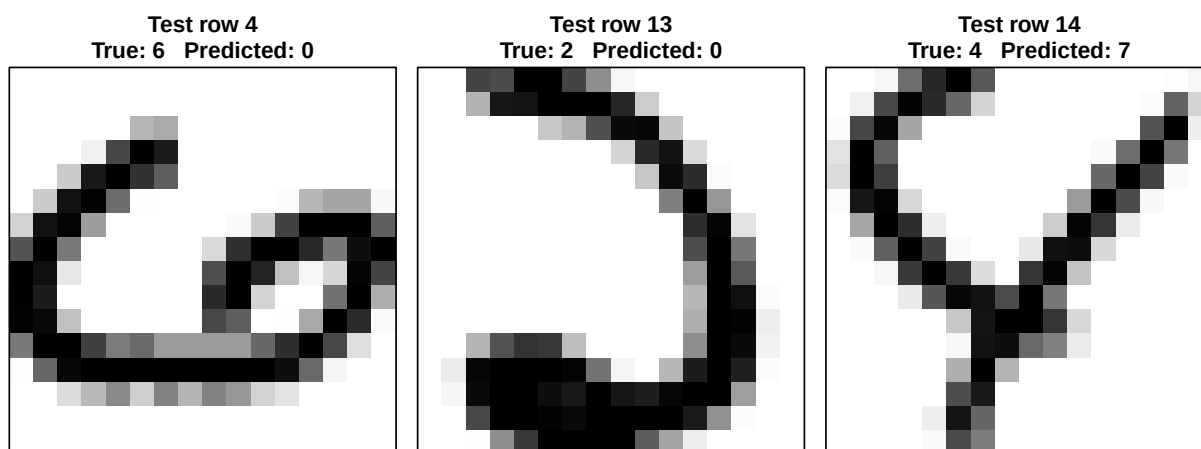


Figure 3: The first three test images misclassified by Euclidean 5NN.

These mistakes are visually understandable. Test row 4 is a 6 with a round lower loop resembling 0; row 13 is a 2 with an unusually curved outline that also resembles 0 in pixel space; and row 14 is a 4 without a strong crossbar, making 7 a plausible nearest-neighbor result. A person can use structural handwriting cues that raw pixel distance does not encode.

The two metrics disagree on 22 test labels. Euclidean is correct while Manhattan is wrong on 11 images, Manhattan is correct while Euclidean is wrong on three, and on eight further images both are wrong but choose different digits. Thus even procedures with fairly similar total error can make meaningfully different mistakes. Euclidean is better on this split, but this single test set does not establish universal superiority.

Question 5: Create a plot-formatting skill

(a) **Design and installation.** I created `format-homework-plots/SKILL.md`. Its description limits it to STAT 432 plot improvements and requires an explicit request. The rules preserve the analysis while covering margins, titles, axes, labels, colors, and figure/font sizes for either R or Python. The supplied validation script stopped before inspecting the file because its Python environment lacks the `yaml` module. I therefore manually verified the required frontmatter, matching folder/name, focused description, and absence of scaffold placeholders, then installed the skill in the normal local Codex skills directory.

(b) **Explicit test on the Question 1 plot.** I explicitly invoked `format-homework-plots` on the original Question 1 plotting code. The simulation values were held fixed.

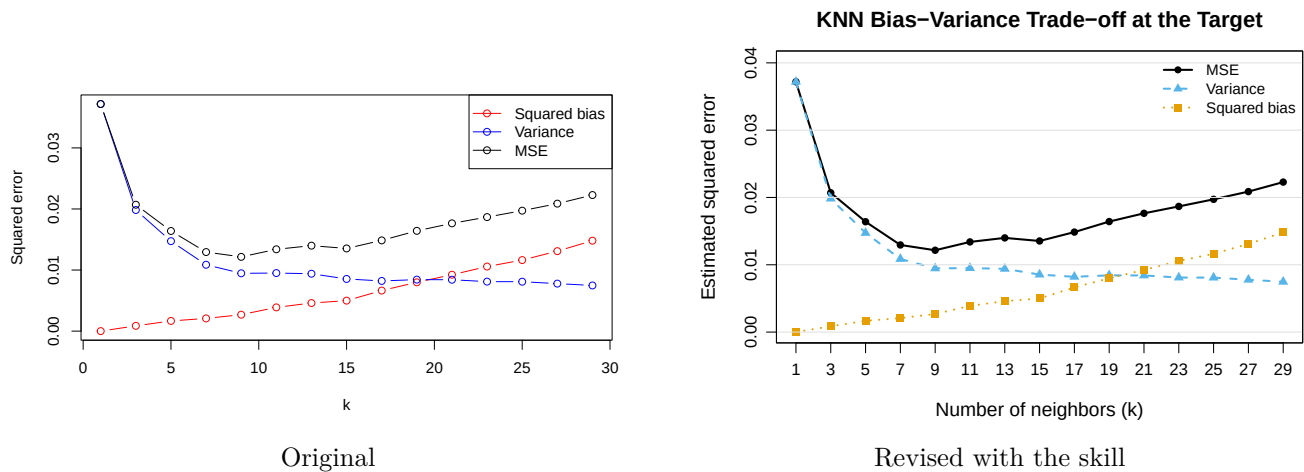


Figure 4: Original and revised renderings of the same Question 1 results.

The revision adds a descriptive title, replaces the vague k label with “Number of neighbors (k),” balances the margins, uses the colorblind-friendly Okabe–Ito blue and orange, and distinguishes curves with markers and line styles in addition to color. It also removes the legend box and uses explicit sizing for the title and labels. I prefer the revised version because the hierarchy and encodings are clearer without changing the statistical message, so I kept the installed skill.

(c) Final SKILL.md.

```

---
name: format-homework-plots
description: Use only when the user explicitly requests this skill to
  improve the formatting of a STAT 432 homework plot.
---

# STAT 432 Plot Formatting

- Preserve the plotted data, calculations, and statistical meaning.
- Use balanced margins so titles, axis labels, tick labels, and legends
  are not clipped.
- Add a short, informative title and clear axis labels, including units
  when relevant.
- Use readable tick spacing and concise labels; avoid unnecessary chart
  decoration.
- Use a colorblind-friendly palette, and distinguish multiple curves
  with markers or line styles as well as color.
- Start with a 6.5 by 4.5 inch figure, 12-point labels, and a 14-point
  title; adjust when a multi-panel layout needs different sizing.
- Apply these rules to the existing R or Python plotting code without
  changing the underlying results.

```